

UniApp StorageSync数据缓存API：全场景实战指南

在UniApp跨平台开发中，数据缓存是实现“离线数据存储”“页面间状态共享”“减少重复请求”的核心技术。StorageSync系列API作为同步式缓存方案，以其用法简洁、无需处理异步回调的特性，成为开发中最常用的缓存工具。本文将系统梳理StorageSync API的核心用法、数据类型支持、实战场景及多端适配技巧，帮助开发者高效、安全地运用数据缓存能力。

一、核心认知：StorageSync与缓存机制

UniApp的缓存体系分为“同步缓存”（StorageSync）和“异步缓存”（Storage），两者功能一致但调用方式不同。StorageSync通过同步方式读写缓存，代码逻辑更线性，适合简单场景；Storage通过异步回调或Promise方式操作，适合大量数据处理，避免阻塞主线程。

1. 核心特性

- **跨页面共享**：缓存数据存储在应用本地，所有页面均可通过API读写，实现页面间数据共享。
- **持久化存储**：除非主动删除或用户清除应用数据，否则缓存数据会永久保留（即使应用重启）。
- **多端兼容**：统一API适配微信小程序、App、H5等多端，底层自动适配各平台存储方案（如小程序的wx.setStorageSync、H5的localStorage）。
- **容量限制**：单条数据大小建议不超过10MB，总缓存容量各平台略有差异（微信小程序约10MB，App端无严格限制但需合理控制）。

2. 适用场景

- 存储用户登录状态（如token、用户ID）。
- 缓存高频访问的静态数据（如地区列表、商品分类）。
- 保存页面临时状态（如表单草稿、筛选条件）。
- 实现离线数据展示（如缓存新闻列表，无网络时显示）。

二、核心API详解：StorageSync完整用法

StorageSync系列API包含“存、取、删、清”四类核心操作，覆盖缓存管理的全流程，所有API均为同步调用，无需处理异步回调。

1. API概览与基础语法

API名称	核心作用	基础语法	关键说明
uni.setStorageSync	存储数据到缓存	uni.setStorageSync(key, data)	key为缓存键名 (字符串类型) data为缓存数据 (任意类型)
uni.getStorageSync	从缓存中读取数据	const data = uni.getStorageSync(key)	key不存在时返回空值 需判断数据有效性
uni.removeStorageSync	删除指定键名的缓存数据	uni.removeStorageSync(key)	仅删除指定key的数据 不影响其他缓存
uni.clearStorageSync	清空所有缓存数据	uni.clearStorageSync()	谨慎使用，会删除所有缓存
uni.getStorageInfoSync	获取缓存信息 (容量、键名列表)	const info = uni.getStorageInfoSync()	返回缓存总容量、当前占用容量、键名数组等信息

2. 数据类型支持与存储规则

StorageSync支持存储字符串、数字、布尔值、对象、数组等多种数据类型，但底层会将非字符串类型自动序列化为JSON字符串，读取时需反序列化。

2.1 支持的核心数据类型

Code block

```
1 // 1. 基础类型：直接存储
2 uni.setStorageSync('username', '张三') // 字符串
3 uni.setStorageSync('age', 25) // 数字
4 uni.setStorageSync('isLogin', true) // 布尔值
5
6 // 2. 复杂类型：自动序列化
7 const userInfo = { id: 1, avatar: 'https://example.com/avatar.png' }
8 uni.setStorageSync('userInfo', userInfo) // 对象
9 uni.setStorageSync('tags', ['前端', 'UniApp', '缓存']) // 数组
10
11 // 3. 读取数据：基础类型直接使用，复杂类型需反序列化
12 const username = uni.getStorageSync('username') // 直接获取: "张三"
13 const userInfo = JSON.parse(uni.getStorageSync('userInfo')) // 反序列化: 对象
14 const tags = JSON.parse(uni.getStorageSync('tags')) // 反序列化: 数组
```

2.2 存储规则与注意事项

- 序列化问题：**存储对象、数组等复杂类型时，UniApp会自动调用 `JSON.stringify` 序列化，但读取时需手动调用 `JSON.parse` 反序列化，否则获取到的是JSON字符串。

- **特殊值处理：** `undefined`、`function`、`Symbol` 等类型无法被JSON序列化，存储时会被忽略或转换为 `null`，需避免存储此类数据。
- **键名唯一性：** 缓存键名 (key) 需唯一，重复存储相同key会覆盖原有数据。
- **容量控制：** 单条数据过大（如超过10MB）会导致存储效率下降，建议拆分存储或使用文件存储 (`uni.saveFile`)。

三、实战场景：StorageSync的核心应用

结合实际业务场景，掌握StorageSync在用户状态管理、数据缓存、离线存储等场景的具体用法。

1. 场景1：用户登录状态管理（token存储）

最核心场景：用户登录后存储token，用于后续接口请求身份验证；退出登录时删除token，清除登录状态。

Code block

```
1  <template>
2    <view class="login-page">
3      <input v-model="username" placeholder="请输入用户名" />
4      <input v-model="password" type="password" placeholder="请输入密码" />
5      <button @click="handleLogin">登录</button>
6      <button @click="handleLogout" type="default">退出登录</button>
7    </view>
8  </template>
9
10 <script setup>
11 import { ref } from 'vue'
12
13 const username = ref('')
14 const password = ref('')
15
16 // 登录逻辑：存储token和用户信息
17 const handleLogin = async () => {
18   try {
19     // 模拟登录接口请求
20     const res = await uni.request({
21       url: 'https://api.example.com/login',
22       method: 'POST',
23       data: { username: username.value, password: password.value }
24     })
25
26     if (res.data.code === 0) {
27       const { token, userInfo } = res.data.data
28       // 1. 存储token（接口请求时需携带）
```

```

29     uni.setStorageSync('token', token)
30     // 2. 存储用户信息（页面展示用）
31     uni.setStorageSync('userInfo', JSON.stringify(userInfo))
32
33     uni.showToast({ title: '登录成功' })
34     // 跳转到首页
35     uni.switchTab({ url: '/pages/home/home' })
36   }
37 } catch (err) {
38   uni.showToast({ title: '登录失败', icon: 'error' })
39 }
40 }
41
42 // 退出登录：清除token和用户信息
43 const handleLogout = () => {
44   // 删除指定缓存
45   uni.removeStorageSync('token')
46   uni.removeStorageSync('userInfo')
47   // 或清空所有缓存（根据需求选择）
48   // uni.clearStorageSync()
49
50   uni.showToast({ title: '已退出登录' })
51   // 跳转到登录页
52   uni.redirectTo({ url: '/pages/login/login' })
53 }
54 </script>

```

2. 场景2：接口数据缓存（减少重复请求）

适用于高频访问且不常变化的接口（如商品分类、地区列表），缓存接口返回数据，指定时间内重复请求时直接读取缓存，提升性能。

Code block

```

1  // 封装带缓存的请求工具函数
2  const requestWithCache = async (options) => {
3    const { url, cacheKey, cacheTime = 3600 } = options // cacheTime单位：秒
4    const now = Date.now()
5
6    // 1. 先从缓存读取数据
7    const cacheData = uni.getStorageSync(cacheKey)
8    if (cacheData) {
9      const { data, timestamp } = JSON.parse(cacheData)
10     // 判断缓存是否过期（当前时间 - 缓存时间 < 缓存有效期）
11     if (now - timestamp < cacheTime * 1000) {
12       console.log('使用缓存数据')

```

```

13     return data // 缓存未过期，返回缓存数据
14   }
15 }
16
17 // 2. 缓存过期或无缓存，发起接口请求
18 try {
19   const res = await uni.request(options)
20   if (res.statusCode === 200 && res.data.code === 0) {
21     // 存储数据到缓存，包含数据和时间戳
22     uni.setStorageSync(cacheKey, JSON.stringify({
23       data: res.data.data,
24       timestamp: now // 记录缓存时间
25     }))
26     console.log('请求接口并缓存数据')
27     return res.data.data
28   }
29 } catch (err) {
30   uni.showToast({ title: '请求失败', icon: 'error' })
31   throw err
32 }
33 }
34
35 // 使用方式：获取商品分类（缓存1小时）
36 const getGoodsCate = async () => {
37   const cateList = await requestWithCache({
38     url: 'https://api.example.com/goods/cate',
39     method: 'GET',
40     cacheKey: 'goodsCateList', // 缓存键名
41     cacheTime: 3600 // 缓存1小时
42   })
43   console.log('商品分类: ', cateList)
44 }

```

3. 场景3：表单草稿保存（防止数据丢失）

适用于长表单页面（如注册、信息编辑），实时保存用户输入的内容到缓存，页面重新进入时恢复草稿，避免用户重复输入。

Code block

```

1 <template>
2   <view class="form-page">
3     <input
4       v-model="formData.name"
5       placeholder="请输入姓名"
6       @input="saveFormDraft"

```

```
7      />
8      <input
9        v-model="formData.phone"
10        type="number"
11        placeholder="请输入手机号"
12        @input="saveFormDraft"
13      />
14      <textarea
15        v-model="formData.intro"
16        placeholder="请输入个人简介"
17        @input="saveFormDraft"
18      ></textarea>
19      <button @click="submitForm">提交表单</button>
20    </view>
21  </template>
22
23  <script setup>
24  import { ref, onLoad } from 'vue'
25
26  const formData = ref({
27    name: '',
28    phone: '',
29    intro: ''
30  })
31
32  // 表单缓存键名（建议包含页面标识，避免冲突）
33  const DRAFT_KEY = 'userEditFormDraft'
34
35  // 页面加载时恢复表单草稿
36  onLoad(() => {
37    const draft = uni.getStorageSync(DRAFT_KEY)
38    if (draft) {
39      formData.value = JSON.parse(draft)
40      uni.showToast({ title: '已恢复表单草稿', icon: 'none' })
41    }
42  })
43
44  // 实时保存表单草稿到缓存
45  const saveFormDraft = () => {
46    // 防抖处理：避免输入时频繁调用缓存API（可选）
47    clearTimeout(window.formTimer)
48    window.formTimer = setTimeout(() => {
49      uni.setStorageSync(DRAFT_KEY, JSON.stringify(formData.value))
50    }, 500)
51  }
52
53  // 提交表单：成功后清除草稿
```

```

54  const submitForm = async () => {
55    try {
56      await uni.request({
57        url: 'https://api.example.com/user/save',
58        method: 'POST',
59        data: formData.value
60      })
61      uni.showToast({ title: '提交成功' })
62      // 提交成功，删除草稿缓存
63      uni.removeStorageSync(DRAFT_KEY)
64      // 跳转页面
65      uni.navigateBack()
66    } catch (err) {
67      uni.showToast({ title: '提交失败', icon: 'error' })
68    }
69  }
70  </script>

```

4. 场景4：缓存信息查询与容量控制

适用于需要展示缓存占用情况或清理缓存的场景（如“设置”页面的缓存清理功能），通过 `getStorageInfoSync` 获取缓存信息。

Code block

```

1  <template>
2    <view class="setting-page">
3      <view class="cache-item">
4        <text>缓存占用: {{ cacheSize }}MB</text>
5        <button @click="clearAllCache" type="default">清理缓存</button>
6      </view>
7      <view class="cache-list">
8        <text>缓存列表: </text>
9        <view v-for="key in cacheKeys" :key="key">{{ key }}</view>
10     </view>
11   </view>
12 </template>
13
14 <script setup>
15 import { ref, onLoad } from 'vue'
16
17 const cacheSize = ref(0)
18 const cacheKeys = ref([])
19
20 // 计算缓存大小（字节转MB）
21 const formatSize = (size) => {

```

```
22     return (size / 1024 / 1024).toFixed(2)
23   }
24
25   // 获取缓存信息
26   const getCacheInfo = () => {
27     const info = uni.getStorageInfoSync()
28     cacheSize.value = formatSize(info.currentSize) // 已用空间（字节）
29     cacheKeys.value = info.keys // 所有缓存键名数组
30     console.log('缓存总容量：', formatSize(info.limitSize), 'MB')
31   }
32
33   onLoad(() => {
34     getCacheInfo()
35   })
36
37   // 清空所有缓存
38   const clearAllCache = () => {
39     uni.showModal({
40       title: '确认清理',
41       content: '是否清空所有缓存？清理后不可恢复。',
42       success: (res) => {
43         if (res.confirm) {
44           uni.clearStorageSync()
45           uni.showToast({ title: '缓存清理完成' })
46           // 重新获取缓存信息
47           getCacheInfo()
48         }
49       }
50     })
51   }
52 </script>
```

四、多端适配与避坑指南

1. 各平台差异与适配方案

平台	差异点	适配方案
微信小程序	1. 缓存容量限制约10MB；2. 同一微信用户、同一小程序的缓存可跨设备同步；3. 开发者工具中可通过“Storage”面板查看缓存	1. 定期清理无用缓存，避免容量超限；2. 敏感数据（如token）建议加密存储；3. 利用getStorageInfoSync监控容量

App端 (iOS/Android)	1. 无严格容量限制，但受设备存储空间影响；2. iOS端缓存存储在沙盒目录，卸载应用会清除；3. Android端部分机型支持SD卡存储	1. 大量数据（如离线数据包）建议用uni.saveFile存储为文件；2. 重要缓存可结合云端备份；3. 提供手动清理缓存功能
H5端	1. 基于浏览器localStorage实现，容量约5MB；2. 跨域名无法共享缓存；3. 清除浏览器缓存会删除应用缓存；4. 部分浏览器隐私模式下缓存不可用	1. 仅存储关键数据，避免大容量缓存；2. 敏感数据不依赖H5缓存；3. 存储前判断浏览器缓存是否可用

2. 常见问题与解决方案

2.1 存储对象/数组后，读取时变成字符串？

原因：StorageSync底层仅支持字符串存储，复杂类型会自动JSON序列化，读取时未反序列化导致获取到JSON字符串。

解决方案：读取复杂类型数据时，必须调用 `JSON.parse()` 反序列化：

Code block

```
1 // 存储
2 const user = { id: 1, name: '张三' }
3 uni.setStorageSync('user', JSON.stringify(user)) // 建议手动序列化，避免隐式转换问题
4
5 // 读取
6 const userStr = uni.getStorageSync('user')
7 const user = userStr ? JSON.parse(userStr) : {} // 反序列化，同时处理空值
```

2.2 缓存数据过期后未自动清理？

原因：StorageSync本身不支持缓存过期自动清理，需手动实现过期判断逻辑。

解决方案：存储数据时关联时间戳，读取时对比当前时间与时间戳，判断是否过期：

Code block

```
1 // 存储带过期时间的数据
2 const setExpireStorage = (key, data, expireSec) => {
3   const storageData = {
4     data,
5     expireTime: Date.now() + expireSec * 1000 // 过期时间戳 (毫秒)
6   }
7   uni.setStorageSync(key, JSON.stringify(storageData))
8 }
```

```

8   }
9
10  // 读取并判断是否过期
11  const getExpireStorage = (key) => {
12    const storageStr = uni.getStorageSync(key)
13    if (!storageStr) return null
14
15    const storageData = JSON.parse(storageStr)
16    // 已过期，删除缓存并返回null
17    if (Date.now() > storageData.expireTime) {
18      uni.removeStorageSync(key)
19      return null
20    }
21    return storageData.data
22  }
23
24  // 使用：存储30分钟过期的数据
25  setExpireStorage('verifyCode', '123456', 1800)
26  // 读取
27  const code = getExpireStorage('verifyCode')

```

2.3 H5端缓存容量不足导致存储失败？

原因：H5端基于localStorage，容量约5MB，超过限制会抛出异常。

解决方案：

拆分大容量数据，仅存储核心字段。存储前判断缓存是否可用及容量是否充足：

Code block

```

1  // 检测H5端缓存是否可用
2  const isStorageAvailable = () => {
3    try {
4      const testKey = 'test_storage_available'
5      uni.setStorageSync(testKey, 'test')
6      uni.removeStorageSync(testKey)
7      return true
8    } catch (err) {
9      return false
10   }
11 }
12
13 // 存储前检测
14 if (isStorageAvailable()) {
15   // 执行存储逻辑
16 } else {
17   uni.showToast({ title: '缓存空间不足，请清理浏览器缓存', icon: 'none' })

```

2.4 敏感数据（如token）存储不安全？

原因：StorageSync存储的数据为明文，App端可通过工具读取，存在安全风险。

解决方案：

加密存储：敏感数据存储前用AES、RSA等算法加密，读取时解密。**App端增强：**使用uni-app的`plus.runtime.permission`获取存储权限，或使用原生插件实现安全存储。**短期存储：**token等临时凭证可存储在内存（如vuex），应用退出后清除，减少本地存储风险。

Code block

```
1  // 简单加密（实际开发需用专业加密库，如crypto-js）
2  const encrypt = (data, key = 'uni_app_secret_key') => {
3    // 模拟加密：简单字符串拼接（仅示例，生产环境禁用）
4    return btoa(JSON.stringify(data) + key)
5  }
6
7  const decrypt = (encryptedData, key = 'uni_app_secret_key') => {
8    const decrypted = atob(encryptedData)
9    // 移除密钥并反序列化
10   return JSON.parse(decrypted.slice(0, -key.length))
11 }
12
13 // 加密存储token
14 const token = 'xxx-xxx-xxx'
15 uni.setStorageSync('token', encrypt(token))
16
17 // 解密读取token
18 const encryptedToken = uni.getStorageSync('token')
19 const token = encryptedToken ? decrypt(encryptedToken) : ''
```

2.5 同一key的缓存在不同页面读写冲突？

原因：多页面同时操作同一key的缓存，可能导致数据覆盖或读取到旧数据。

解决方案：

键名规范化：键名包含页面标识或模块标识（如“user_edit_form”“goods_detail_123”）。

读写加锁：通过全局变量标记缓存是否正在操作，避免并发冲突（适用于复杂场景）。

五、总结

StorageSync系列API是UniApp中最简单高效的数据缓存方案，核心优势在于同步调用、用法简洁、多端兼容。使用时需把握以下核心原则：

1. **数据类型适配**：复杂类型（对象、数组）需手动序列化/反序列化，避免读取到字符串。
2. **容量与安全控制**：H5端控制缓存大小，敏感数据加密存储，避免明文暴露。
3. **场景化设计**：登录状态用removeStorageSync清除，接口数据加时间戳控制过期，表单草稿实时保存。
4. **多端适配**：针对小程序、App、H5的差异，调整存储策略（如H5不存大容量数据）。

实际开发中，建议封装缓存工具类，统一处理序列化、加密、过期判断等逻辑，提升代码可维护性。同时，合理区分缓存场景，避免过度依赖缓存导致数据一致性问题——缓存适用于非实时、高频访问的数据，实时性要求高的数据（如实时订单状态）应直接请求接口。

（注：文档部分内容可能由 AI 生成）